

# DOCUMENTO MAESTRO — NAVI HERRAMIENTAS & ACTIVOS / FENIX365

## 1. Propósito de este documento

Este documento consolida todo lo trabajado hasta el momento sobre el proyecto **NAVI Herramientas & Activos**, también referido como **NAVI Herramientas**, **Fenix365 Herramientas** o **Herramientas & Activos**.

La finalidad es dejar un contexto completo, claro y suficientemente profundo para iniciar una nueva conversación de desarrollo sin perder el historial de decisiones, necesidades, avances, problemas detectados, arquitectura, base de datos, roles, permisos, módulos funcionales, pendientes y próximos pasos.

Este documento debe ser leído junto con el código fuente completo del proyecto, la auditoría de base de datos, los prototipos HTML, la cotización inicial, las transcripciones de reuniones, el proceso de compras MRO, los archivos Excel de hojas de vida y los documentos técnicos que han sido entregados durante el desarrollo.

El objetivo no es solamente explicar qué se ha construido, sino también dejar claro **por qué se está construyendo, cuál es la necesidad real, qué se ha logrado, qué riesgos existen, qué falta validar, qué falta desarrollar y cómo debe continuar el proyecto.**

## 2. Contexto general del proyecto

### 2.1 Nombre del proyecto

Nombre recomendado:

**NAVI Herramientas & Activos**

También puede identificarse internamente como:

- NAVI Herramientas
- Fenix365 Herramientas
- Herramientas & Activos
- Gestión de Herramientas y Activos de Taller
- Complemento de Activos Fijos / Herramientas para Fenix365

El nombre funcional sugerido para el documento maestro es:

**Documento Maestro Funcional y Técnico — NAVI Herramientas & Activos**

### 2.2 Organización y contexto operativo

El proyecto se desarrolla para el contexto operativo de **Navitrans**, con énfasis en la gestión de herramientas, activos de taller, equipos, accesorios, evidencias, hojas de vida técnicas, mantenimiento, préstamos, asignaciones, tomas físicas y conciliación con información proveniente de Fenix365 / Dynamics 365.

La solución no debe comportarse como una aplicación aislada sin relación con la operación. Debe actuar como una extensión funcional y visual del ecosistema de Navitrans, especialmente de Fenix365 / Dynamics 365.

Desde el inicio se ha definido que la solución debe respetar:

- La lógica de operación por sede.

- La lógica de operación por zona.
- La trazabilidad empresarial.
- La matriz de roles y permisos.
- La gestión documental.
- La auditoría de cambios.
- El control de herramientas por responsable.
- La hoja de vida técnica de cada herramienta.
- La conciliación con información de Fenix365 / Dynamics.
- El uso móvil para técnicos, herramenteros y participantes de tomas físicas.
- El uso web administrativo para administración, coordinadores, ingenieros, herramenteros y gerencia.

### **3. Necesidad principal del proyecto**

#### **3.1 Necesidad de negocio**

Navitrans requiere una solución que permita gestionar de forma centralizada, trazable y controlada todo el ciclo de vida de las herramientas y activos de taller. Actualmente, la información puede estar distribuida entre hojas de vida en Excel, exportaciones de Dynamics/Fenix365, documentos adjuntos, evidencias, correos, registros manuales, información de técnicos, registros de mantenimiento y procesos no completamente centralizados.

La necesidad principal es construir una plataforma que permita:

1. Saber qué herramientas existen.
2. Saber dónde están.
3. Saber en qué sede están.
4. Saber a qué responsable o técnico están asignadas.
5. Saber en qué estado operativo están.
6. Saber si están disponibles, prestadas, asignadas, en mantenimiento, dañadas, dadas de baja o pendientes por revisar.
7. Consultar su hoja de vida técnica.
8. Registrar documentos, evidencias y fichas técnicas.
9. Registrar mantenimientos.
10. Registrar daños o novedades.
11. Gestionar préstamos y devoluciones.
12. Gestionar asignaciones fijas.
13. Realizar tomas físicas.
14. Comparar información física contra información administrativa/Fenix365.
15. Controlar qué puede hacer cada usuario según su rol y sede.
16. Evitar que usuarios sin permiso puedan ver, crear, aprobar, modificar, eliminar o conciliar información.
17. Preparar el sistema para una futura integración controlada con Dynamics/Fenix365.

### 3.2 Problema actual identificado

El problema no es únicamente "tener inventario". El problema real es la falta de un sistema unificado que conecte:

- Inventario.
- Activos fijos.
- Herramientas.
- Responsables.
- Técnicos.
- Sedes.
- Zonas.
- Hojas de vida.
- Mantenimientos.
- Evidencias.
- Compras MRO.
- Préstamos.
- Devoluciones.
- Tomas físicas.
- Conciliación administrativa.
- Trazabilidad.
- Seguridad por permisos.

Sin una solución centralizada, se presentan riesgos como:

- Duplicidad de herramientas.
- Herramientas sin responsable claro.
- Herramientas registradas en Excel pero no conciliadas con Fenix365.
- Herramientas registradas en Fenix365 pero no encontradas físicamente.
- Hojas de vida incompletas.
- Evidencias dispersas.
- Mantenimientos sin trazabilidad.
- Compras o solicitudes por correo sin control suficiente.
- Usuarios con permisos excesivos.
- Usuarios sin claridad sobre qué pueden hacer.
- Falta de control por sede.
- Falta de control por zona.
- Dificultad para auditar acciones.
- Dificultad para generar reportes confiables.
- Dificultad para operar desde móvil en taller o campo.

### 4. Objetivo general

Diseñar, desarrollar e implementar una solución web administrativa y móvil PWA para la gestión integral de herramientas y activos de taller, permitiendo controlar inventario, hoja de vida técnica, documentos, evidencias, mantenimiento,

préstamos, asignaciones, tomas físicas, novedades, conciliación con Fenix365 y gestión de roles/permisos, con trazabilidad completa y seguridad restrictiva por perfil, sede y alcance funcional.

## **5. Objetivos específicos**

### **5.1 Objetivos funcionales**

1. Permitir la creación, consulta, edición y control de herramientas y activos.
2. Clasificar herramientas por tipo, categoría, sede, zona, ubicación y responsable.
3. Registrar hoja de vida técnica por herramienta.
4. Asociar accesorios a una herramienta.
5. Asociar prácticas seguras a una herramienta.
6. Asociar documentos, evidencias y fichas técnicas.
7. Registrar eventos de ciclo de vida.
8. Registrar daños y novedades.
9. Gestionar mantenimientos.
10. Gestionar solicitudes de mantenimiento.
11. Gestionar préstamos y devoluciones.
12. Gestionar asignaciones fijas de activos a responsables o retorno a almacén/taller.
13. Gestionar tomas físicas por sede, zona, responsable o participante.
14. Permitir reporte de herramientas encontradas, faltantes, dañadas o diferentes.
15. Permitir conciliación de registros físicos contra inventario administrativo.
16. Controlar solicitudes de compra relacionadas con herramientas o activos.
17. Preparar el módulo MRO para trazabilidad de compras, cotizaciones y órdenes.
18. Generar reportes y dashboards administrativos.
19. Permitir operación móvil para técnicos, herramenteros y participantes.
20. Administrar usuarios, roles y permisos.
21. Auditar acciones críticas.
22. Permitir carga/importación desde hojas de vida y exportaciones de Fenix365.
23. Preparar la solución para integraciones futuras.

### **5.2 Objetivos técnicos**

1. Construir una solución basada en .NET 8.
2. Usar arquitectura por capas.
3. Separar API, Admin Web, Mobile PWA, Domain, Application, Infrastructure, Shared y Worker.

4. Usar SQL Server como base de datos principal.
5. Usar Docker para levantar servicios locales.
6. Usar MinIO para almacenamiento de documentos y evidencias.
7. Usar Hangfire para procesos en segundo plano.
8. Usar Redis si aplica para cache o procesos auxiliares.
9. Mantener Swagger para documentación y pruebas de API.
10. Implementar autenticación y autorización.
11. Preparar el sistema para JWT y Microsoft Entra ID.
12. Usar permisos restrictivos por endpoint y por menú.
13. Mantener auditoría sobre acciones críticas.
14. Crear scripts de respaldo, restauración y diagnóstico de base de datos.
15. Permitir migrar el proyecto a otro PC usando Docker, backup SQL y respaldo de MinIO.

## **6. Alcance funcional definido**

### **6.1 Módulos principales**

El sistema debe contemplar los siguientes módulos:

1. Dashboard.
2. Inventario de herramientas y activos.
3. Detalle de herramienta.
4. Hoja de vida técnica.
5. Accesorios.
6. Prácticas seguras.
7. Documentos y evidencias.
8. Disponible y ubicación.
9. Asignación de activo fijo.
10. Historial de asignación.
11. Préstamos y devoluciones.
12. Daños y novedades.
13. Mantenimiento.
14. Solicitudes de mantenimiento.
15. Planes de mantenimiento.
16. Consulta de mantenimiento.
17. Solicitudes de compra.
18. Compras MRO.
19. Importación de consolidado de hojas de vida.
20. Importación de export de Dynamics/Fenix365.
21. Conciliación.
22. Toma física.
23. Detalle de toma física.
24. Registros reportados por usuario.

25. Ubicaciones, sedes, zonas y responsables.
26. Configuración de catálogos.
27. Usuarios.
28. Roles y permisos.
29. Reportes.
30. Vista móvil / PWA.
31. Vista previa móvil en web.
32. Integración Fenix365.
33. Auditoría.
34. Scripts de diagnóstico, respaldo y restauración.

## **7. Alcance contractual vs alcance real**

### **7.1 Alcance inicial de cotización**

La cotización inicial define una plataforma web para:

- Gestión de inventario.
- Hoja de vida de herramienta.
- Mantenimiento.
- Préstamos y asignaciones.
- Gestión de técnicos.
- Novedades y daños.
- Solicitud de compra.
- Reportes y dashboards.
- Gestión de usuarios y roles.

Ese alcance se concentra principalmente en una plataforma web operativa.

### **7.2 Alcance real ampliado durante el análisis**

Durante el análisis se amplió la necesidad real del sistema. Ahora se contempla:

- Aplicación web administrativa.
- Aplicación móvil PWA.
- Conciliación contra Fenix365.
- Importación de hojas de vida.
- Importación de exportaciones de Dynamics/Fenix365.
- Operación por sedes.
- Operación por zonas.
- Auditoría.
- Documentos en MinIO.
- Procesos en segundo plano con Hangfire.
- Seguridad restrictiva por permisos.
- Roles por perfil operativo.
- Módulos móviles visibles según permisos.
- MRO y compras relacionadas con herramientas/mantenimiento.

- Toma física avanzada.
- Registros reportados por usuario.
- Preparación para integración futura con Fenix365.

### **7.3 Recomendación sobre alcance**

Se recomienda separar el proyecto en fases:

#### **Fase 1 — MVP operativo interno**

Debe permitir operar internamente:

- Login.
- Usuarios.
- Roles.
- Permisos.
- Dashboard básico.
- Inventario.
- Hoja de vida.
- Documentos.
- Accesorios.
- Prácticas seguras.
- Asignaciones.
- Disponible y ubicación.
- Préstamos básicos.
- Toma física.
- Reportes básicos.
- Importación.
- Conciliación básica.
- PWA móvil básica.
- Auditoría mínima.

#### **Fase 2 — Control avanzado**

Debe fortalecer:

- Mantenimiento.
- Solicitudes de mantenimiento.
- Compras MRO.
- Flujo de aprobación.
- Ordenamiento de permisos por sede/zona.
- Alertas.
- Reportes ejecutivos.
- Validación de calidad de datos.
- Documentación técnica.
- Seguridad avanzada.

#### **Fase 3 — Integración controlada con Fenix365/Dynamics**

Debe contemplar:

- Sincronización controlada.
- Lectura de información de Fenix365.
- Conciliación automática.
- Publicación de novedades si aplica.
- Envío de solicitudes de compra si se aprueba funcionalmente.
- Envío de estados o bajas si se aprueba funcionalmente.
- Jobs programados.
- Bitácora de integración.
- Manejo de errores y reintentos.

## **8. Arquitectura técnica definida**

### **8.1 Stack principal**

El stack definido es:

- .NET 8.
- ASP.NET Core Web API.
- Blazor Server para Admin Web.
- Blazor WebAssembly / PWA para móvil.
- SQL Server.
- Entity Framework Core.
- Docker.
- MinIO.
- Hangfire.
- Redis.
- Swagger.
- JWT.
- Microsoft Entra ID en una fase posterior.
- Git/GitHub.
- Azure DevOps si aplica.

### **8.2 Estructura de solución**

La solución está organizada con los siguientes proyectos:

Navitrans.ToolsAssets.Management.sln

```
src/
  Navi.ToolsAssets.Api/
  Navi.ToolsAssets.Admin/
  Navi.ToolsAssets.MobilePwa/
  Navi.ToolsAssets.Application/
  Navi.ToolsAssets.Domain/
  Navi.ToolsAssets.Infrastructure/
  Navi.ToolsAssets.Shared/
  Navi.ToolsAssets.Worker/
```



## 8.3 Responsabilidad por proyecto

### **Navi.ToolsAssets.Api**

Contiene los controladores HTTP, endpoints REST, autenticación, autorización, documentación Swagger y conexión con la capa de infraestructura.

Controladores principales identificados:

- AuthController.
- CatalogsController.
- DamagesController.
- DashboardController.
- ExecutiveDashboardController.
- ImportsController.
- LoansController.
- MaintenanceController.
- MaintenanceRequestsController.
- OrganizationController.
- PhysicalCountsController.
- PurchaseRequestsController.
- ReconciliationController.
- SettingsManagementController.
- TechnicalLifeRecordsController.
- ToolAccessoriesController.
- ToolAssetControlController.
- ToolDocumentsController.
- ToolsController.
- SecurityRolesEduardoController.
- Controladores de demo/dev que deben revisarse antes de producción.

### **Navi.ToolsAssets.Admin**

Contiene la aplicación web administrativa en Blazor Server.

Páginas principales identificadas:

- Login.
- Home / Dashboard.
- AssetsIndex.
- AssetAvailability.
- AssetAssignment.
- AssetAssignmentHistory.
- TechnicalLifeRecord.
- DocumentsIndex.
- AccessoriesIndex.
- MaintenanceIndex.
- MaintenancePlans.
- MaintenanceRequest.

- MaintenanceConsultation.
- LoansIndex.
- DamagesIndex.
- PhysicalCountDetail.
- MroPurchasesIndex.
- ImportConsolidated.
- ImportDynamics.
- FenixIntegration.
- CatalogsSettings.
- OrganizationSettings.
- MobilePreview.
- Approvals.
- DisposalsIndex.
- Users/Roles desde Settings o Seguridad.

### **Navi.ToolsAssets.MobilePwa**

Contiene la PWA móvil.

Este frente aún está en desarrollo. Se ha hablado de trabajar el menú móvil por partes. Actualmente el foco estaba en:

- Menú Historial.
- Menú Hoja de Vida.
- Menú Mis Herramientas.
- Menú Reportar daño.
- Menú Toma física.
- Menú Evidencias.
- Menú Pendientes.
- Menú Perfil.
- Visibilidad de opciones según rol y permiso.

### **Navi.ToolsAssets.Domain**

Contiene entidades del negocio:

- AppRole.
- AppUser.
- Branch.
- Zone.
- ToolLocation.
- ResponsiblePerson.
- ToolAsset.
- ToolType.
- ToolCategory.
- ToolAccessory.
- ToolSafePractice.
- ToolDocument.
- ToolLifeCycleEvent.

- ToolLoan.
- ToolLoanItem.
- DamageReport.
- MaintenanceRecord.
- MaintenanceRequest.
- PhysicalCount.
- PhysicalCountItem.
- PhysicalCountParticipant.
- PhysicalCountReportedItem.
- PhysicalCountExtraItem.
- PurchaseRequest.
- ImportBatch.
- ImportRow.
- FenixReconciliationRecord.
- SystemParameter.
- SettingCatalogItem.

### **Navi.ToolsAssets.Infrastructure**

Contiene:

- DbContext.
- Configuraciones EF Core.
- Migraciones.
- Seed de datos.
- Repositorios si aplica.
- MinIO storage.
- Seguridad JWT/Entra.
- Reportes.
- Servicios de infraestructura.

### **Navi.ToolsAssets.Application**

Debe contener reglas de aplicación, servicios de caso de uso, validaciones, orquestación de procesos y lógica no dependiente directamente de UI.

### **Navi.ToolsAssets.Shared**

Actualmente debe revisarse porque se detectó que puede estar poco usado o vacío. Se recomienda usarlo para DTOs compartidos entre API, Admin y Mobile PWA, siempre que no cree acoplamiento peligroso.

### **Navi.ToolsAssets.Worker**

Debe usarse para procesos de segundo plano:

- Sincronización.
- Jobs de mantenimiento.
- Alertas.
- Conciliación programada.
- Limpieza.

- Integración Fenix365.
- Reportes programados.

## 9. Infraestructura local con Docker

### 9.1 Servicios usados

El ambiente local usa contenedores Docker para:

- SQL Server.
- MinIO.
- Redis.

#### SQL Server

Base de datos:

NaviToolsAssetsDb

Puerto usado en el ambiente probado:

localhost,1434

El contenedor interno usa SQL Server, pero el puerto publicado puede cambiar según docker-compose. En el equipo actual se confirmó conexión exitosa con localhost,1434.

#### MinIO

Se usa para documentos y evidencias.

Bucket esperado:

navi-tools-documents

#### Redis

Puede usarse para cache, sesiones o soporte futuro. No es crítico para migrar datos si todavía no almacena información indispensable.

### 9.2 Migración a otro PC

Para abrir el proyecto en otro PC con Docker se debe mover:

1. Código fuente.
2. Archivo .env o local.env.example adaptado.
3. Backup .bak de SQL Server.
4. Respaldo de MinIO si existen documentos/evidencias.
5. Instrucciones de restauración.
6. No copiar bin, obj, .vs, node\_modules, ni archivos temporales.

Pasos recomendados:

1. Clonar o copiar el proyecto.
2. Instalar Docker Desktop.
3. Instalar .NET 8 SDK.
4. Instalar Git.
5. Levantar contenedores con docker compose up -d.
6. Restaurar backup de SQL Server.
7. Restaurar MinIO o volver a cargar evidencias si son pocas.
8. Ejecutar API.

9. Ejecutar Admin.
10. Probar login.
11. Probar dashboard.
12. Probar inventario.
13. Probar roles/permisos.
14. Probar PWA móvil.

## **10. Estado actual de la base de datos**

### **10.1 Base de datos auditada**

La base auditada es:

NaviToolsAssetsDb

Se generó un documento de auditoría llamado:

NAVI\_Documento\_Seguridad\_Completo.txt

La auditoría permitió observar tablas, columnas, relaciones, roles, usuarios y parte de la matriz de seguridad.

### **10.2 Tablas y registros detectados**

La base tiene información real y no está vacía.

Datos relevantes:

- AppRoles: 7 registros.
- AppUsers: 76 registros.
- ToolDocuments: 2 registros.
- ImportBatches: 1 registro.
- ImportRows: 2 registros.
- ToolAccessories: 420 registros.
- ToolAssets: 185 registros.
- ToolCategories: 14 registros.
- ToolTypes: 7 registros.
- ToolLifeCycleEvents: 423 registros.
- ToolLoanItems: 2 registros.
- ToolLoans: 2 registros.
- MaintenanceRecords: 0 registros.
- MaintenanceRequests: 0 registros.
- Branches: 15 registros.
- ResponsiblePeople: 80 registros.
- SettingCatalogItems: 14 registros.
- SystemParameters: 5 registros.
- ToolLocations: 47 registros.
- Zones: 6 registros.
- PhysicalCountItems: 2 registros.
- PhysicalCountParticipants: 12 registros.
- PhysicalCountReportedItems: 14 registros.

- PhysicalCounts: 2 registros.
- PurchaseRequests: 0 registros.
- ToolSafePractices: 225 registros.
- FenixReconciliationRecords: 0 registros.

Esto indica que:

1. Inventario ya tiene datos.
2. Accesorios ya tiene datos.
3. Prácticas seguras ya tiene datos.
4. Ciclo de vida ya tiene eventos.
5. Organización ya tiene sedes, responsables, ubicaciones y zonas.
6. Seguridad ya tiene usuarios y roles.
7. Toma física ya tiene registros de prueba.
8. Mantenimiento todavía está vacío.
9. Compras todavía está vacío.
10. Conciliación Fenix todavía está vacía.

### 10.3 Hallazgo importante en auditoría SQL

El script de auditoría inicialmente falló en una sección porque intentó consultar:

`dbo.Branches`

Pero la tabla real está en:

`Organization.Branches`

También aplica para:

`Organization.Zones`

`Organization.ToolLocations`

`Organization.ResponsiblePeople`

Por lo tanto, los scripts futuros no deben asumir que todo está en `dbo`.

Recomendación:

1. Usar nombres con esquema correcto.
2. Consultar `sys.schemas` y `sys.tables`.
3. No hardcodear `dbo` para tablas organizacionales, inventario, documentos, mantenimiento, préstamos, compras, daños, seguridad o sincronización.
4. Ajustar scripts de auditoría y documentación para leer automáticamente el esquema real.

## 11. Modelo de datos funcional

### 11.1 Seguridad

Tablas principales:

- AppRoles.
- AppUsers.

#### AppRoles

Campos relevantes:

- Id.
- Code.
- Name.
- Description.
- Permissions.
- IsActive.
- CreatedAt.
- CreatedBy.
- UpdatedAt.
- UpdatedBy.
- IsDeleted.

Actualmente Permissions es un texto que puede guardar permisos separados por coma o punto y coma, según los datos existentes.

Riesgo detectado:

Hay inconsistencia entre separadores:

- Algunos roles tienen permisos separados por coma.
- Otros roles tienen permisos separados por punto y coma.

Esto es crítico porque si el código solo divide por ,, los roles con permisos separados por coma pueden quedar interpretados como un solo permiso largo y no funcionar correctamente. Si el código solo divide por coma, ocurrirá lo contrario.

Recomendación:

Crear un método centralizado de parseo de permisos que soporte:

;

,

|

saltos de línea

espacios

Y usarlo en:

- API.
- Admin.
- Mobile PWA.
- Scripts de auditoría.
- Seed de roles.
- Login.
- Middleware/filters de permisos.

## **AppUsers**

Campos relevantes:

- Id.
- UserName.
- DisplayName.
- Email.

- Position.
- Area.
- AppRoleId.
- BranchId.
- ResponsiblePersonId.
- IsActive.
- LastLoginAt.
- CreatedAt.
- CreatedBy.
- UpdatedAt.
- UpdatedBy.
- IsDeleted.
- PasswordHash.

La relación principal es:

`AppUsers.AppRoleId → AppRoles.Id`

También se relaciona con:

`BranchId → Organization.Branches.Id`

`ResponsiblePersonId → Organization.ResponsiblePeople.Id`

## 11.2 Organización

Tablas principales:

- Organization.Zones.
- Organization.Branches.
- Organization.ToolLocations.
- Organization.ResponsiblePeople.
- Organization.SystemParameters.
- Organization.SettingCatalogItems.

### Zones

Representan zonas operativas.

### Branches

Representan sedes.

Campos importantes:

- Code.
- Name.
- City.
- Address.
- ZoneId.
- IsPilot.
- IsActive.
- IsDeleted.

### ToolLocations

Representan ubicaciones, almacenes o talleres dentro de una sede.



## **ResponsiblePeople**

Representan responsables, técnicos, herramienteros, ingenieros, coordinadores o personas asociadas a activos.

### **11.3 Inventario**

Tablas principales:

- Inventory.ToolAssets.
- Inventory.ToolAccessories.
- Inventory.ToolTypes.
- Inventory.ToolCategories.

#### **ToolAssets**

Es la tabla principal de activos/herramientas.

Campos relevantes:

- InternalCode.
- Name.
- Description.
- Brand.
- Model.
- SerialNumber.
- FixedAssetCode.
- FenixCode.
- AcquisitionDate.
- UsefulLifeMonths.
- UnitOfMeasure.
- Quantity.
- RequiresMaintenance.
- RequiresPreOperationalCheck.
- RequiresCertification.
- CertificationExpirationDate.
- Zoneld.
- BranchId.
- LocationId.
- ResponsiblePersonId.
- ToolTypeId.
- ToolCategoryId.
- OperationalStatus.
- PhysicalStatus.
- CustodyStatus.
- ReconciliationStatus.
- SyncStatus.
- FenixName.
- FenixStatus.

- FenixBranch.
- FenixResponsible.
- LastSyncAt.
- IsSpecialized.
- HasWarranty.
- LastMaintenanceDate.
- LoadCapacity.
- MaintenancePeriodMonths.
- NextMaintenanceDate.
- Provider.
- UsefulLifeDays.
- UsefulLifeStartDate.
- Voltage.
- WarrantyType.

Este modelo ya permite manejar activos físicos, herramientas, información técnica, estado, responsable, sede, ubicación, mantenimiento, garantía, sincronización y conciliación.

## 11.4 Documentos

Tabla principal:

- Documents.ToolDocuments.

Sirve para almacenar referencias a documentos guardados en MinIO.

Campos relevantes:

- ToolAssetId.
- DocumentType.
- FileName.
- ObjectKey.
- ContentType.
- SizeBytes.
- UploadedBy.
- UploadedAt.
- Description.

Importante:

La base de datos guarda metadata del archivo, pero el archivo físico vive en MinIO. Si se mueve el proyecto a otro PC, también se debe mover o reconstruir MinIO.

## 11.5 Ciclo de vida

Tabla principal:

- LifeCycle.ToolLifeCycleEvents.

Sirve para registrar eventos como:

- Creación.
- Edición.

- Cambio de estado.
- Asignación.
- Devolución.
- Mantenimiento.
- Daño.
- Documento.
- Baja.
- Conciliación.
- Importación.
- Sincronización.

Campos relevantes:

- ToolAssetId.
- EventType.
- Title.
- Description.
- PreviousValue.
- NewValue.
- RegisteredBy.
- RegisteredAt.

## 11.6 Préstamos

Tablas principales:

- Loans.ToolLoans.
- Loans.ToolLoanItems.

Sirven para registrar préstamos de herramientas, sus ítems y devoluciones.

## 11.7 Mantenimiento

Tablas principales:

- Maintenance.MaintenanceRecords.
- Maintenance.MaintenanceRequests.

Actualmente están creadas, pero sin registros.

Esto indica que el módulo existe en modelo, pero falta validación funcional fuerte y carga real de datos.

## 11.8 Compras

Tabla principal:

- Purchases.PurchaseRequests.

Actualmente está creada, pero sin registros.

Esto indica que el módulo de compras existe en modelo, pero debe alinearse con el proceso MRO real y todavía requiere mayor definición funcional y pruebas.

## 11.9 Toma física

Tablas principales:

- PhysicalCounts.PhysicalCounts.
- PhysicalCounts.PhysicalCountItems.
- PhysicalCounts.PhysicalCountParticipants.
- PhysicalCounts.PhysicalCountReportedItems.
- PhysicalCounts.PhysicalCountExtraItems.

Estas tablas permiten:

- Crear una toma física.
- Generar participantes.
- Registrar ítems esperados.
- Registrar ítems contados.
- Registrar ítems reportados por usuarios.
- Registrar ítems extra encontrados.
- Gestionar conciliación de hallazgos.
- Aprobar creación de activos desde hallazgos.
- Rechazar reportes.
- Pedir aclaración al usuario.
- Cerrar toma física.

## 11.10 Conciliación

Tabla principal:

- Sync.FenixReconciliationRecords.

Actualmente está vacía.

Debe usarse para comparar:

- Inventario físico.
- Datos internos.
- Export de Fenix365.
- Código de activo fijo.
- Código Fenix.
- Estado Fenix.
- Sede Fenix.
- Responsable Fenix.
- Diferencias.
- Resultado del proceso.

## 12. Roles actuales detectados

Actualmente existen 7 roles:

1. ADMIN.
2. COORD\_TALLER.
3. GERENCIAL.
4. HERRAMIENTERO.

5. ING\_SERVICIOS.
6. TECNICO.
7. TECNICO B.

## **12.1 ADMIN**

Rol de acceso total.

Debe poder:

- Ver dashboard.
- Gestionar inventario.
- Crear herramientas.
- Editar herramientas.
- Eliminar o desactivar herramientas.
- Gestionar disponibilidad.
- Gestionar asignaciones.
- Ver historial.
- Gestionar documentos.
- Gestionar mantenimiento.
- Gestionar compras.
- Gestionar tomas físicas.
- Ver reportes.
- Gestionar configuración.
- Gestionar usuarios.
- Gestionar roles.
- Usar móvil.
- Ver herramientas móviles.
- Revisar herramientas móviles.
- Reportar daño móvil.
- Solicitar préstamos móviles.
- Reportar preoperacional móvil.
- Gestionar bajas si aplica.
- Generar acta de entrega si aplica.

Riesgo:

El rol ADMIN mezcla permisos existentes en catálogo con permisos nuevos o móviles que aún deben formalizarse en SecurityPermissions.

## **12.2 COORD\_TALLER**

Rol de coordinador de taller.

Debe poder:

- Consultar dashboard.
- Ver inventario.
- Crear activos/herramientas.
- Editar activos/herramientas.

- Ver y editar disponibilidad.
- Ver asignaciones.
- Asignar activos.
- Retornar activos.
- Ver hoja de vida.
- Editar hoja de vida.
- Ver documentos.
- Subir documentos.
- Ver mantenimiento.
- Solicitar mantenimiento.
- Ejecutar mantenimiento.
- Cerrar mantenimiento.
- Ver compras.
- Solicitar compras.
- Aprobar compras.
- Ver tomas físicas.
- Crear tomas físicas.
- Cerrar tomas físicas.
- Ver reportes.
- Acceso móvil básico.
- Ver herramientas desde móvil.

Debe revisarse si realmente debe aprobar compras. Si el coordinador también crea solicitudes, debe evitarse la autoaprobación sin control.

## **12.3 GERENCIAL**

Rol de consulta ejecutiva y aprobación.

Debe poder:

- Ver dashboard.
- Ver inventario.
- Ver hoja de vida.
- Ver compras.
- Aprobar compras.
- Ver mantenimiento.
- Ver tomas físicas.
- Ver reportes.

No debería:

- Crear activos.
- Editar activos.
- Eliminar activos.
- Gestionar usuarios.
- Gestionar roles.
- Cambiar configuración.

- Ejecutar mantenimiento.
- Hacer conciliación operativa si no aplica.
- Modificar datos maestros.

## **12.4 HERRAMIENTERO**

Rol de gestión operativa de herramientas.

Debe poder:

- Ver dashboard.
- Ver inventario.
- Crear herramientas si su operación lo requiere.
- Editar herramientas si su operación lo requiere.
- Ver disponibilidad.
- Editar disponibilidad.
- Ver asignaciones.
- Asignar herramientas.
- Retornar herramientas.
- Ver historial de asignaciones.
- Ver hoja de vida.
- Editar hoja de vida.
- Ver documentos.
- Subir documentos.
- Descargar documentos.
- Ver mantenimiento.
- Solicitar mantenimiento.
- Ver tomas físicas.
- Crear tomas físicas.
- Cerrar tomas físicas.
- Ver reportes.
- Usar móvil.
- Ver herramientas en móvil.
- Revisar herramientas en móvil.

Debe revisarse si puede crear y cerrar toma física o si solo debe participar/reportar.

## **12.5 ING\_SERVICIOS**

Rol de ingeniero de servicios.

Debe poder:

- Ver dashboard.
- Ver inventario.
- Editar activos si aplica.
- Ver disponibilidad.
- Ver asignaciones.

- Ver hoja de vida.
- Ver documentos.
- Subir documentos.
- Ver mantenimiento.
- Solicitar mantenimiento.
- Ejecutar mantenimiento.
- Cerrar mantenimiento.
- Ver compras.
- Aprobar compras.
- Ver tomas físicas.
- Ver reportes.
- Acceso móvil.
- Ver herramientas en móvil.

Debe revisarse si puede aprobar compras. Puede tener aprobación técnica, pero no necesariamente aprobación administrativa.

## 12.6 TECNICO

Rol de técnico.

Debe poder principalmente desde móvil:

- Acceder a la PWA.
- Ver sus herramientas.
- Ver hoja de vida relacionada.
- Ver documentos relacionados.
- Revisar herramienta.
- Reportar daño.
- Reportar preoperacional.
- Solicitar préstamo si el flujo lo permite.
- Solicitar mantenimiento si el flujo lo permite.
- Participar en toma física asignada.
- Ver historial propio.

No debe:

- Crear activos.
- Editar activos administrativos.
- Eliminar activos.
- Aprobar compras.
- Rechazar compras.
- Conciliar.
- Aprobar creación de activos desde toma física.
- Gestionar usuarios.
- Gestionar roles.
- Gestionar configuración.
- Ver toda la información global.



- Ver reportes administrativos globales.
- Modificar sedes o ubicaciones.

## 12.7 TECNICO B

Rol de prueba o rol temporal.

Actualmente parece tener muy pocos permisos:

- Maintenance.View.
- TechnicalLifeRecord.View.
- Tools.View.

Debe revisarse si este rol es real o fue creado como prueba.

Si es prueba, debe eliminarse o marcarse como inactivo antes de producción.

## 13. Catálogo de permisos actual esperado

El catálogo esperado en código incluye:

- Dashboard.View.
- Tools.View.
- Tools.Create.
- Tools.Edit.
- Tools.Delete.
- AssetAvailability.View.
- AssetAvailability.Edit.
- AssetAssignment.View.
- AssetAssignment.Assign.
- AssetAssignment.Return.
- AssetAssignment.History.
- TechnicalLifeRecord.View.
- TechnicalLifeRecord.Edit.
- TechnicalLifeRecord.Export.
- Documents.View.
- Documents.Upload.
- Documents.Download.
- Documents.Delete.
- Maintenance.View.
- Maintenance.Request.
- Maintenance.Execute.
- Maintenance.Close.
- Purchases.View.
- Purchases.Request.
- Purchases.Approve.
- Purchases.Reject.
- PhysicalCounts.View.
- PhysicalCounts.Create.

- PhysicalCounts.Close.
- SafePractices.View.
- SafePractices.Manage.
- Reports.View.
- Reconciliation.View.
- Reconciliation.Manage.
- Settings.View.
- Settings.Manage.
- Security.Users.
- Security.Roles.

#### **14. Permisos adicionales detectados en base o conversación**

Además del catálogo esperado, se han usado o mencionado permisos como:

- Mobile.Access.
- Mobile.Tools.View.
- Mobile.Tools.Review.
- Mobile.Damage.Report.
- Mobile.Loans.Request.
- Mobile.PreOperational.Report.
- DeliveryAct.Generate.
- FixedAssets.Disposal.Request.
- FixedAssets.Disposal.Approve.

Estos permisos deben formalizarse en el catálogo de seguridad.

Recomendación:

Agregar al catálogo SecurityPermissions los permisos móviles y de baja/actas si realmente van a existir en el sistema.

Propuesta de permisos móviles:

Mobile.Access

Mobile.Tools.View

Mobile.Tools.Review

Mobile.History.View

Mobile.TechnicalLifeRecord.View

Mobile.Documents.View

Mobile.Documents.Upload

Mobile.Damage.Report

Mobile.Loans.Request

Mobile.PreOperational.Report

Mobile.PhysicalCounts.View

Mobile.PhysicalCounts.Report

Mobile.PhysicalCounts.Finish

Mobile.Profile.View

Propuesta de permisos de baja:

FixedAssets.Disposal.View

FixedAssets.Disposal.Request  
FixedAssets.Disposal.Approve  
FixedAssets.Disposal.Reject  
FixedAssets.Disposal.Execute  
Propuesta de permisos de acta:  
DeliveryAct.View  
DeliveryAct.Generate  
DeliveryAct.Download

## **15. Problemas detectados en seguridad**

### **15.1 Separadores inconsistentes en permisos**

Hay permisos separados por coma y otros separados por punto y coma.  
Esto puede romper:

- Login.
- Menús.
- Validación de permisos.
- Endpoints API.
- Auditoría.
- Móvil.
- Scripts de seguridad.

Acción requerida:

Crear un método único:

ParsePermissions(string permissions)

Debe soportar:

- ;
- ,
- |
- saltos de línea
- espacios
- duplicados
- mayúsculas/minúsculas

### **15.2 Catálogo de permisos incompleto frente a base**

El rol ADMIN y otros roles tienen permisos móviles y funcionales que no aparecen en el catálogo esperado inicial.

Acción requerida:

Actualizar SecurityPermissions.All con todos los permisos reales.

### **15.3 Permisos web y móvil mezclados**

Actualmente algunos roles tienen permisos móviles, pero la matriz no está completamente formalizada.

Acción requerida:

Definir permisos por canal:

- Web.
- Mobile.
- API.
- Jobs.
- Integración.

## 15.4 Usuarios y roles por sede

Hay usuarios por sede:

- coordinador.agu
- coordinador.ame
- coordinador.arr
- etc.
- herramientero.agu
- herramientero.ame
- ingeniero.agu
- tecnico1.agu
- tecnico2.agu
- etc.

Esto indica que la seguridad debe filtrar información por sede.

Acción requerida:

Verificar que todos los endpoints respeten el alcance del usuario:

- ADMIN: total.
- GERENCIAL: total o por zona según definición.
- COORD\_TALLER: sede o zona.
- ING\_SERVICIOS: sede o zona.
- HERRAMIENTERO: sede.
- TECNICO: propio / asignado.

## 15.5 Endpoints sin validación fuerte

Se debe revisar cada controlador para validar:

- Tiene autorización.
- Tiene permiso requerido.
- Valida usuario activo.
- Valida rol activo.
- Valida sede/alcance.
- No permite modificación por usuario sin permiso.
- No expone información sensible.
- No permite acciones administrativas desde móvil sin permiso.

## 15.6 Control de menús no suficiente

Ocultar un menú en Blazor o PWA no basta. También debe protegerse el endpoint

API.

Regla:

Si un usuario no tiene permiso, no debe ver el menú, no debe ver el botón y tampoco debe poder ejecutar el endpoint manualmente.

## **16. Módulos web desarrollados o avanzados**

### **16.1 Dashboard**

Estado:

Avanzado.

Debe mostrar:

- Total herramientas.
- Disponibles.
- Asignadas.
- En mantenimiento.
- Con daño.
- Inconsistencias.
- Pendientes de toma física.
- Solicitudes pendientes.
- Mantenimientos vencidos.
- Herramientas sin hoja de vida.
- Documentos pendientes.

Pendiente:

- Validar permisos.
- Validar filtros por sede/zona.
- Validar KPIs contra datos reales.
- Validar rendimiento.

### **16.2 Inventario de activos**

Estado:

Muy avanzado.

Ya existe:

- Listado.
- Filtros.
- Estados.
- Relación con sede.
- Relación con ubicación.
- Relación con responsable.
- Tipo.
- Categoría.
- Datos Fenix.
- Datos de mantenimiento.

- Datos de garantía.
- Estado de custodia.
- Estado físico.
- Estado operativo.
- Estado de conciliación.

Pendiente:

- Validar crear.
- Validar editar.
- Validar eliminar lógico.
- Validar búsqueda.
- Validar filtro por sede.
- Validar que técnico solo vea lo suyo.
- Validar que herramientero vea sede.
- Validar que administrador vea todo.

### **16.3 Disponible y ubicación**

Estado:

Avanzado.

Objetivo:

Permitir cambiar ubicación, sede, almacén, taller o disponibilidad.

Pendiente:

- Validar que solo usuarios autorizados puedan cambiar ubicación.
- Registrar evento en ciclo de vida.
- Validar consistencia con responsable.
- Validar si un activo asignado puede moverse o no.
- Validar reglas cuando está prestado, en mantenimiento o dado de baja.

### **16.4 Asignación de activo fijo**

Estado:

Avanzado con corrección aplicada o pendiente de confirmar.

Se detectó un problema importante:

El endpoint de asignar/devolver tenía doble atributo de permiso. Esto podía bloquear usuarios con permiso de retorno si no tenían también permiso de asignación.

La lógica correcta es:

- Para asignar a responsable: requiere AssetAssignment.Assign.
- Para regresar a almacén/taller: requiere AssetAssignment.Return.

Pendiente:

- Confirmar que el patch quedó aplicado.
- Compilar solución.
- Probar con usuario que solo tenga Assign.
- Probar con usuario que solo tenga Return.

- Probar con usuario sin permisos.
- Verificar ciclo de vida.
- Verificar historial.

## **16.5 Hoja de vida técnica**

Estado:

Avanzado.

Debe incluir:

- Información técnica.
- Marca.
- Modelo.
- Serial.
- Código interno.
- Código activo fijo.
- Código Fenix.
- Estado.
- Mantenimiento.
- Vida útil.
- Garantía.
- Documentos.
- Accesorios.
- Prácticas seguras.
- Historial de eventos.
- Exportación a Excel o PDF.

Pendiente:

- Validar permisos.
- Validar exportación.
- Validar que móvil pueda ver hoja de vida según permiso.
- Validar que técnico solo vea información permitida.
- Validar que edición sea restringida.

## **16.6 Documentos**

Estado:

Avanzado.

Usa MinIO.

Debe permitir:

- Cargar documentos.
- Descargar documentos.
- Asociarlos a herramienta.
- Asociarlos a mantenimiento.
- Asociarlos a evidencia.
- Registrar metadatos.

Pendiente:

- Validar permisos de upload/download/delete.
- Validar que al migrar PC también se migre MinIO.
- Validar extensión/tamaño.
- Validar seguridad de ObjectKey.
- Validar no exposición de rutas internas.

## **16.7 Accesorios**

Estado:

Avanzado.

La base tiene muchos accesorios cargados.

Pendiente:

- Validar alta/edición.
- Validar relación con herramienta.
- Validar si accesorio requiere mantenimiento.
- Validar si accesorio es equipo de medición.
- Validar móvil si aplica.

## **16.8 Prácticas seguras**

Estado:

Avanzado.

La base tiene prácticas seguras.

Pendiente:

- Validar gestión por tipo de herramienta.
- Validar visualización en hoja de vida.
- Validar visualización móvil.
- Validar permisos de administración.

## **16.9 Préstamos**

Estado:

Modelo y módulos base existen.

La base tiene algunos registros de préstamo.

Pendiente:

- Validar flujo completo.
- Solicitud.
- Aprobación.
- Entrega.
- Devolución.
- Condición de entrega.
- Condición de devolución.
- Registro en hoja de vida.
- Restricción por disponibilidad.



- Restricción por daño.
- Restricción por mantenimiento.
- Permisos móviles.

## **16.10 Daños y novedades**

Estado:

Modelo existe.

Pendiente:

- Validar reporte web.
- Validar reporte móvil.
- Validar evidencia.
- Validar si bloquea préstamo.
- Validar si cambia estado operativo.
- Validar si dispara mantenimiento.
- Validar si genera evento.

## **16.11 Mantenimiento**

Estado:

Modelo creado, pero sin datos reales en la auditoría.

Debe contemplar:

- Solicitud.
- Programación.
- Ejecución.
- Cierre.
- Evidencia.
- Costo.
- Proveedor.
- Técnico.
- Resultado.
- Actualización de hoja de vida.
- Cambio de estado de herramienta.
- Próxima fecha de mantenimiento.

Pendiente:

- Validar módulo completo.
- Validar solicitud de mantenimiento.
- Validar aprobación.
- Validar ejecución.
- Validar cierre.
- Validar roles.
- Validar integración con MRO si implica compra o servicio.
- Validar notificaciones o alertas.

## **16.12 Compras / MRO**

Estado:

Modelo base creado, pero sin datos reales en PurchaseRequests.

Debe alinearse con el proceso MRO documentado.

## **17. Proceso MRO documentado**

### **17.1 Necesidad**

Compras MRO debe centralizar y controlar solicitudes de compra relacionadas con:

- Herramientas.
- Equipos.
- Mantenimiento.
- Dotación.
- Papelería.
- Cafetería.
- Aseo.
- Suvenir.
- EPP.
- Botiquín.
- Material de empaque.
- Accesorios TI.
- Equipos de oficina.
- Muebles.
- Reparaciones locativas.
- Servicios de mantenimiento.

### **17.2 Problema actual MRO**

Hay procesos que se hacen por correo. Esto genera:

- Baja trazabilidad.
- Cotizaciones dispersas.
- Aprobaciones no estructuradas.
- Órdenes de compra creadas después de la factura.
- Falta de conexión con base de datos.
- Dificultad para auditar.
- Dificultad para saber si el código/variante existe.
- Dependencia de correos y soportes manuales.

### **17.3 Proceso objetivo MRO**

El flujo recomendado:

1. El solicitante identifica necesidad.
2. Crea solicitud de compra.

3. Define motivo MRO.
4. Registra líneas.
5. Indica código, variante, cantidad, especificaciones, almacén y dimensiones.
6. Si no existe código o variante, se solicita creación con planificación.
7. Envía solicitud a flujo.
8. Aprobador revisa.
9. Compras MRO valida datos.
10. Compras MRO valida cuenta/concepto.
11. Compras MRO cotiza.
12. Se registran cotizaciones.
13. Se selecciona proveedor.
14. Se autoriza opción.
15. Compras MRO crea orden de compra.
16. Se recibe bien o servicio.
17. Se procesa factura.
18. Se cierra proceso con soportes.

#### **17.4 Controles MRO**

Debe validarse:

- Tipo de compra.
- Monto.
- Responsable.
- Canal de entrada.
- Código.
- Variante.
- Cuenta contable.
- Concepto.
- Especificaciones.
- Fotos.
- Serial.
- Falla.
- Tipo de mantenimiento.
- Cotizaciones.
- Aprobación.
- Orden de compra previa.
- Recepción.
- Factura.
- Cierre.

## **17.5 MRO pendiente**

El sistema debe definir si herramientas y mantenimiento entran por:

1. Módulo de mantenimiento.
2. Solicitud de compra.
3. Flujo mixto.
4. Actividad técnica + compra.
5. Integración futura con Fenix365.

Recomendación:

Para el MVP, crear una trazabilidad interna mínima en NAVI:

- Solicitud.
- Tipo de necesidad.
- Activo relacionado.
- Sede.
- Responsable.
- Justificación.
- Estado.
- Aprobación.
- Cotización adjunta o referencia.
- Observaciones.
- Número de OC si existe.
- Estado en Dynamics/Fenix si se obtiene luego.

## **18. Toma física**

### **18.1 Necesidad**

La toma física debe permitir validar que las herramientas registradas existen físicamente, están en la sede correcta, están en la ubicación correcta y están bajo el responsable correcto.

### **18.2 Funciones esperadas**

1. Crear toma física.
2. Seleccionar sede.
3. Seleccionar responsables/participantes.
4. Generar ítems esperados.
5. Asignar participantes.
6. Iniciar toma física.
7. Registrar herramientas encontradas.
8. Registrar herramientas no encontradas.
9. Registrar herramientas dañadas.
10. Registrar herramientas diferentes.
11. Registrar herramientas extra.
12. Solicitar aclaración al usuario.
13. Aprobar creación de activo.

14. Rechazar reporte.
15. Conciliar.
16. Cerrar toma física.
17. Generar reporte.

### **18.3 Estado actual**

Ya existen tablas y registros de prueba para:

- PhysicalCounts.
- PhysicalCountItems.
- PhysicalCountParticipants.
- PhysicalCountReportedItems.

Esto indica que el módulo está avanzado, pero falta validación exhaustiva.

## **19. PWA móvil**

### **19.1 Necesidad móvil**

La operación de taller requiere que técnicos, herramenteros y participantes puedan actuar desde celular.

La PWA debe permitir:

- Ver herramientas asignadas.
- Ver hoja de vida.
- Ver documentos.
- Ver historial.
- Reportar daño.
- Reportar novedad.
- Reportar preoperacional.
- Solicitar préstamo si aplica.
- Participar en toma física.
- Subir evidencia con cámara.
- Ver pendientes.
- Operar según permisos.

### **19.2 Menús móviles propuestos**

Menú móvil base:

1. Inicio.
2. Mis herramientas.
3. Hoja de vida.
4. Historial.
5. Reportar daño.
6. Evidencias.
7. Toma física.
8. Pendientes.
9. Préstamos.

- 10. Preoperacional.
- 11. Perfil.
- 12. Sincronización.

### **19.3 Regla fundamental**

El menú móvil debe comportarse según permisos.

Ejemplos:

- Si no tiene Mobile.Access, no entra a la PWA.
- Si no tiene Mobile.Tools.View, no ve Mis herramientas.
- Si no tiene Mobile.Tools.Review, no puede revisar herramienta.
- Si no tiene Mobile.Damage.Report, no puede reportar daño.
- Si no tiene Mobile.Loans.Request, no puede solicitar préstamo.
- Si no tiene Mobile.PreOperational.Report, no puede reportar preoperacional.
- Si no tiene TechnicalLifeRecord.View, no ve hoja de vida.
- Si no tiene Documents.Upload, no sube evidencias.
- Si no tiene PhysicalCounts.View, no ve tomas físicas.

### **19.4 Estado actual del móvil**

Se había definido avanzar por partes.

El foco actual estaba en:

- Historial.
- Hoja de vida.
- Menú móvil por permisos.

Pendiente:

- Confirmar rutas.
- Confirmar componentes.
- Confirmar DTOs.
- Conectar API real.
- Validar login.
- Validar sesión.
- Validar permisos.
- Validar offline si aplica.
- Validar carga de evidencias.
- Validar cámara.
- Validar QR si se implementa.
- Validar filtros por usuario.

## **20. Seguridad requerida**

### **20.1 Principio rector**

La seguridad debe basarse en el principio de mínimo privilegio.

Cada usuario solo debe ver y hacer lo que su rol, sede, zona y responsabilidad

permitan.

## **20.2 Niveles de seguridad**

Se deben aplicar controles en:

1. Login.
2. Sesión.
3. Menú web.
4. Menú móvil.
5. Botones.
6. Formularios.
7. Endpoints API.
8. Consultas SQL.
9. Filtros por sede.
10. Filtros por usuario.
11. Filtros por responsable.
12. Auditoría.
13. Documentos.
14. Integración.
15. Jobs.

## **20.3 Matriz de seguridad esperada**

La matriz debe responder:

- Qué rol puede ver cada módulo.
- Qué rol puede crear.
- Qué rol puede editar.
- Qué rol puede eliminar.
- Qué rol puede aprobar.
- Qué rol puede rechazar.
- Qué rol puede cerrar.
- Qué rol puede conciliar.
- Qué rol puede exportar.
- Qué rol puede operar en móvil.
- Qué rol opera por sede.
- Qué rol opera por zona.
- Qué rol opera global.
- Qué rol solo ve información propia.

## **20.4 Acciones críticas que requieren permiso explícito**

Deben requerir permisos separados:

- Crear activo.
- Editar activo.
- Eliminar activo.

- Cambiar ubicación.
- Asignar activo.
- Retornar activo.
- Crear préstamo.
- Aprobar préstamo.
- Devolver préstamo.
- Reportar daño.
- Cerrar daño.
- Solicitar mantenimiento.
- Ejecutar mantenimiento.
- Cerrar mantenimiento.
- Solicitar compra.
- Aprobar compra.
- Rechazar compra.
- Crear toma física.
- Cerrar toma física.
- Aprobar creación desde toma física.
- Rechazar reporte de toma física.
- Conciliar.
- Cargar documento.
- Eliminar documento.
- Exportar hoja de vida.
- Administrar usuarios.
- Administrar roles.
- Administrar configuración.

## **21. Auditoría requerida**

Toda acción importante debe registrar:

- Usuario.
- Rol.
- Fecha.
- Acción.
- Entidad afectada.
- Id de entidad.
- Valor anterior.
- Valor nuevo.
- Sede.
- Origen: Web, móvil, job, importación.
- IP o dispositivo si aplica.
- Resultado.
- Observación.

Eventos auditables:



- Creación.
- Edición.
- Eliminación lógica.
- Importación.
- Conciliación.
- Aprobación.
- Rechazo.
- Préstamo.
- Devolución.
- Daño.
- Mantenimiento.
- Baja.
- Documento adjunto.
- Sincronización.
- Login.
- Error de autorización.
- Cambio de rol.
- Cambio de permisos.

## **22. Lo que ya se hizo**

### **22.1 Documentación y análisis**

Ya se trabajó:

- Análisis de necesidad.
- Documento maestro inicial.
- Cotización funcional.
- Proceso MRO.
- Diagramas MRO.
- Prototipos HTML de Admin y Mobile.
- Análisis de roles.
- Matriz de permisos.
- Scripts de auditoría de base de datos.
- Auditoría de tablas, roles y usuarios.
- Revisión de infraestructura Docker.
- Guía para migrar a otro PC.
- Revisión de problema de asignación/retorno.
- Patch para permiso de asignación/retorno.
- Revisión de seguridad inicial.

### **22.2 Código**

Ya existe una solución avanzada con:

- API.

- Admin.
- Mobile PWA.
- Domain.
- Infrastructure.
- Application.
- Shared.
- Worker.
- Docker.
- SQL Server.
- MinIO.
- Hangfire.
- Redis.
- Migraciones EF.
- Seed de datos.
- Controladores principales.
- Páginas Admin.
- Modelos de dominio.
- Base de datos poblada.

### **22.3 Base de datos**

Ya existen:

- Roles.
- Usuarios.
- Sedes.
- Zonas.
- Responsables.
- Ubicaciones.
- Herramientas.
- Accesorios.
- Categorías.
- Tipos.
- Documentos.
- Eventos de ciclo de vida.
- Prácticas seguras.
- Toma física.
- Algunos préstamos.
- Scripts de auditoría.

### **22.4 Seguridad**

Ya existe:

- Tabla de roles.
- Tabla de usuarios.

- Campo de permisos por rol.
- Permisos definidos en código.
- Roles base.
- Usuarios por sede.
- Usuarios por perfil.
- Usuarios por responsable.
- Login.
- Control de permisos en frontend.
- Control parcial en API.
- Patch o revisión para asignar/retornar activos.

## **23. Lo que falta**

### **23.1 Validación técnica**

Falta:

1. Compilar solución completa.
2. Ejecutar API.
3. Ejecutar Admin.
4. Ejecutar Mobile PWA.
5. Validar conexión SQL.
6. Validar MinIO.
7. Validar Redis.
8. Validar Hangfire.
9. Validar Swagger.
10. Ejecutar migraciones en ambiente limpio.
11. Restaurar backup en otro PC.
12. Validar que el proyecto corre desde cero.

### **23.2 Validación funcional web**

Falta probar exhaustivamente:

- Dashboard.
- Login.
- Usuarios.
- Roles.
- Inventario.
- Crear herramienta.
- Editar herramienta.
- Eliminar herramienta.
- Disponible y ubicación.
- Asignar activo.
- Retornar activo.
- Historial.

- Hoja de vida.
- Exportación.
- Documentos.
- Accesorios.
- Prácticas seguras.
- Préstamos.
- Daños.
- Mantenimiento.
- Solicitudes de mantenimiento.
- Compras.
- Toma física.
- Conciliación.
- Importaciones.
- Reportes.

### **23.3 Validación de seguridad**

Falta hacer una revisión endpoint por endpoint:

1. Qué permiso exige.
2. Qué rol lo tiene.
3. Qué menú lo llama.
4. Qué botón lo ejecuta.
5. Qué datos devuelve.
6. Si filtra por sede.
7. Si filtra por responsable.
8. Si filtra por usuario.
9. Si permite acciones cruzadas.
10. Si permite acceso móvil sin permiso.
11. Si permite modificar desde Postman sin permiso.
12. Si registra auditoría.

### **23.4 Validación de base de datos**

Falta:

1. Corregir scripts que asumen dbo para tablas de otros esquemas.
2. Generar documento completo de base sin errores.
3. Exportar matriz de roles y permisos.
4. Exportar matriz usuario/rol/permiso.
5. Verificar permisos desconocidos.
6. Verificar permisos no asignados.
7. Verificar usuarios sin rol.
8. Verificar roles sin permisos.
9. Verificar usuarios inactivos.
10. Verificar roles de prueba.

11. Verificar campos sensibles.
12. Verificar índices.
13. Verificar relaciones.
14. Verificar constraints.
15. Verificar backup/restore.

## **23.5 Móvil PWA**

Falta:

1. Definir menú final.
2. Definir permisos móviles.
3. Formalizar permisos en catálogo.
4. Crear pantalla inicio móvil.
5. Crear Mis herramientas.
6. Crear Hoja de vida móvil.
7. Crear Historial móvil.
8. Crear Reportar daño.
9. Crear Evidencias.
10. Crear Toma física móvil.
11. Crear Pendientes.
12. Crear Perfil.
13. Validar login móvil.
14. Validar sesión.
15. Validar permisos.
16. Validar carga de imagen.
17. Validar offline si aplica.
18. Validar sincronización.
19. Validar responsive.
20. Validar navegación.

## **23.6 MRO**

Falta:

1. Definir matriz de decisión por tipo de compra.
2. Definir umbrales.
3. Definir responsables.
4. Definir si herramientas/mantenimiento entra por solicitud de compra o mantenimiento.
5. Definir campos adicionales.
6. Definir flujo de aprobación.
7. Definir cotizaciones.
8. Definir OC.
9. Definir relación con Dynamics.
10. Crear registros de prueba.

11. Validar permisos.
12. Validar que no exista autoaprobación sin control.
13. Validar trazabilidad.

## **23.7 Documentación**

Falta:

1. Documento funcional maestro final.
2. Documento técnico de arquitectura.
3. Documento de base de datos.
4. Documento de seguridad.
5. Documento de roles y permisos.
6. Manual de usuario Admin.
7. Manual de usuario Mobile.
8. Guía de despliegue local.
9. Guía de backup/restore.
10. Guía de migración a otro PC.
11. Presentación ejecutiva.
12. Diagrama de arquitectura.
13. Diagrama de módulos.
14. Diagrama de flujo MRO.
15. Diagrama de seguridad.
16. Diagrama de toma física.
17. Diagrama de integración Fenix365.

## **24. Riesgos detectados**

### **24.1 Riesgo de alcance**

El alcance real creció más allá de la cotización inicial.

Mitigación:

Separar MVP, fase 2 e integración Fenix365.

### **24.2 Riesgo de seguridad**

Permisos inconsistentes o incompletos pueden permitir acceso indebido.

Mitigación:

Crear matriz formal, normalizar permisos y validar API.

### **24.3 Riesgo de datos**

Hay datos en diferentes esquemas. Scripts que asumen dbo fallan.

Mitigación:

Usar esquema real o detección automática.

### **24.4 Riesgo de contraseñas**

Se han usado credenciales locales durante pruebas.

Mitigación:

No subir local.env a Git, rotar claves y usar secretos.

## **24.5 Riesgo de MinIO**

La base guarda metadata, pero si MinIO no se migra, se pierden archivos físicos.

Mitigación:

Respaldar bucket navi-tools-documents.

## **24.6 Riesgo de permisos móviles**

La PWA puede mostrar menús sin que API esté protegida.

Mitigación:

Proteger menú, botón y endpoint.

## **24.7 Riesgo de autoaprobación**

Roles como coordinador o ingeniero pueden tener permisos de solicitud y aprobación.

Mitigación:

Implementar regla que impida aprobar una solicitud creada por el mismo usuario, salvo excepción explícita.

## **24.8 Riesgo de datos reales vs demo**

Existen controladores y datos de demo/dev.

Mitigación:

Eliminar o proteger endpoints demo antes de producción.

## **25. Reglas de diseño funcional**

### **25.1 Reglas de inventario**

- Todo activo debe tener código interno.
- Todo activo debe tener nombre.
- Todo activo debe tener sede.
- Todo activo debe tener zona.
- Todo activo debe tener estado operativo.
- Todo activo debe tener estado físico.
- Todo activo debe tener estado de custodia.
- Todo activo puede tener responsable.
- Todo activo puede tener ubicación.
- Todo activo puede tener tipo.
- Todo activo puede tener categoría.
- Todo activo puede tener código Fenix.
- Todo activo puede tener código de activo fijo.

## **25.2 Reglas de asignación**

- Solo usuarios con AssetAssignment.Assign pueden asignar.
- Solo usuarios con AssetAssignment.Return pueden retornar.
- Toda asignación debe registrar evento.
- Todo retorno debe registrar evento.
- No se debe asignar un activo dado de baja.
- No se debe asignar un activo bloqueado por daño crítico.
- No se debe asignar un activo en mantenimiento si no está operativo.
- El historial debe quedar consultable.

## **25.3 Reglas de hoja de vida**

- La hoja de vida debe consolidar datos técnicos, administrativos y operativos.
- Debe mostrar accesorios.
- Debe mostrar prácticas seguras.
- Debe mostrar documentos.
- Debe mostrar eventos.
- Debe mostrar mantenimientos.
- Debe mostrar préstamos.
- Debe mostrar daños.
- Debe mostrar cambios de estado.
- Debe permitir exportación si el usuario tiene permiso.

## **25.4 Reglas de documentos**

- Todo documento debe estar asociado a una entidad.
- El archivo físico debe guardarse en MinIO.
- La metadata debe guardarse en SQL.
- Debe registrarse quién subió el documento.
- Debe registrarse fecha.
- Debe validarse permiso de descarga.
- Debe validarse permiso de eliminación.

## **25.5 Reglas de toma física**

- Solo usuarios con permiso pueden crear toma física.
- Los participantes solo deben ver lo que les corresponde.
- Los reportes deben quedar asociados al participante.
- Los extras deben poder pasar a conciliación.
- La creación de activos desde extras debe requerir aprobación.
- La toma no debe cerrarse si hay pendientes críticos, salvo excepción definida.
- Toda decisión debe quedar auditada.



## 25.6 Reglas de MRO

- Toda compra debe tener necesidad.
- Toda compra debe tener solicitante.
- Toda compra debe tener sede.
- Toda compra debe tener justificación.
- Toda compra debe tener estado.
- Toda aprobación debe quedar registrada.
- Toda cotización debe quedar relacionada.
- La OC debe ser previa a la compra/factura.
- No se deben legalizar compras tardíamente sin trazabilidad.
- Se debe definir cuándo aplica compra local vs Compras MRO.

## 26. Próximo orden de trabajo recomendado

### Paso 1 — Cerrar documento maestro

Tomar este documento como base y complementarlo con:

- Capturas.
- Tablas reales.
- Matriz completa.
- Diagramas.
- Reglas por rol.

### Paso 2 — Corregir auditoría SQL

Crear script que lea esquemas reales:

- dbo.AppRoles.
- dbo.AppUsers.
- Organization.Branches.
- Organization.Zones.
- Organization.ToolLocations.
- Organization.ResponsiblePeople.
- Inventory.ToolAssets.
- etc.

Generar:

- Documento de base.
- Documento de seguridad.
- CSV de roles.
- CSV de usuarios.
- CSV de matriz usuario/rol/permiso.
- CSV de permisos desconocidos.
- CSV de permisos no asignados.

### **Paso 3 — Normalizar permisos**

Corregir:

- Separadores de permisos.
- Catálogo incompleto.
- Permisos móviles.
- Permisos de baja.
- Permisos de actas.
- Permisos de conciliación granular.
- Permisos de toma física granular.

### **Paso 4 — Revisar código de seguridad**

Revisar:

- RequirePermission.
- HasPermission.
- HasAnyPermission.
- AuthSession.
- NaviPermissionHttpRequestHandler.
- Controllers.
- NavMenu.
- MobilePwa menu.
- Endpoints sensibles.

### **Paso 5 — Validar web Admin**

Probar cada módulo con:

- ADMIN.
- COORD\_TALLER.
- HERRAMIENTERO.
- ING\_SERVICIOS.
- GERENCIAL.
- TECNICO.

### **Paso 6 — Continuar PWA móvil**

Prioridad:

1. Login móvil.
2. Menú por permiso.
3. Mis herramientas.
4. Hoja de vida.
5. Historial.
6. Reportar daño.
7. Evidencias.
8. Toma física.

- 9. Pendientes.
- 10. Perfil.

## **Paso 7 — MRO**

Definir y desarrollar:

- Matriz MRO.
- Flujo de solicitud.
- Aprobación.
- Cotización.
- OC.
- Relación con mantenimiento.
- Relación con herramienta.
- Estado.
- Documentos.
- Reportes.

## **Paso 8 — Documentación final**

Crear:

- Documento maestro funcional.
- Documento técnico.
- Documento de seguridad.
- Documento base de datos.
- Manual Admin.
- Manual Mobile.
- Presentación ejecutiva.

## **27. Prompt recomendado para nuevo chat**

Usar este texto al iniciar un nuevo chat:

Estoy desarrollando el proyecto NAVI Herramientas & Activos para Navitrans. Es una solución .NET 8 con Web API, Blazor Server Admin, Blazor WebAssembly/PWA móvil, SQL Server, Docker, MinIO, Hangfire y Redis. El objetivo es gestionar herramientas y activos de taller: inventario, hoja de vida técnica, documentos, accesorios, prácticas seguras, mantenimiento, préstamos, asignaciones, daños, tomas físicas, conciliación con Fenix365/Dynamics, compras MRO, usuarios, roles, permisos, auditoría y operación móvil.

El sistema debe comportarse como una extensión funcional y visual de Fenix365/Dynamics, pero separando MVP interno, fase de control avanzado e integración futura. Ya existe código avanzado con proyectos Api, Admin, MobilePwa, Application,

Domain, Infrastructure, Shared y Worker. La base de datos se llama NaviToolsAssetsDb y tiene datos reales: roles, usuarios, sedes, zonas, responsables, ubicaciones, herramientas, accesorios, eventos, documentos, prácticas seguras y tomas físicas.

Necesito continuar desde este contexto, revisando primero seguridad, usuarios, roles, permisos y matriz restrictiva. Luego continuar con PWA móvil, especialmente menús Historial y Hoja de Vida, respetando permisos. Después continuar con MRO, documento maestro y presentación.

No asumir que todas las tablas están en dbo. Hay esquemas como Organization, Inventory, Documents, LifeCycle, Loans, Maintenance, PhysicalCounts, Purchases, Safety y Sync. Hay que revisar código y base antes de modificar.

## 28. Conclusión

El proyecto ya no está en etapa conceptual. Ya existe una base técnica y funcional avanzada.

Lo más importante ahora no es seguir agregando módulos sin control, sino estabilizar:

1. Seguridad.
2. Roles.
3. Permisos.
4. Matriz por usuario.
5. Alcance por sede.
6. Alcance móvil.
7. Consistencia de base de datos.
8. Scripts de respaldo/auditoría.
9. Compilación.
10. Pruebas por módulo.

Después de eso, el camino correcto es continuar con la PWA móvil y luego cerrar MRO, documentación maestra y presentación ejecutiva.

La regla principal para continuar debe ser:

**Primero seguridad y trazabilidad; después nuevas funcionalidades.**